

CREATOR PLATFORM

Technical & Product Blueprint

Pre-Documentation for Functional Demo, Investment Approval and Product Expansion

Current objective

Build a functional proof of concept that demonstrates the core value proposition: one media asset can be scheduled and distributed to multiple creator platforms from a single system. The demo should be technically credible, visually professional, private, and ready to evolve into a production SaaS after investment approval.

Status snapshot - September 2026

Project base created locally • Private GitHub repository connected • Multilingual routing working • Premium login UI created
• Neon PostgreSQL connected • Prisma 8 contract workflow validated

Document Map

#	Section
1	Executive Summary
2	Product Vision & Core Problem
3	Proof of Concept Scope
4	Target Users and Multi-Tenant Model
5	Core User Flows
6	Technology Stack
7	System Architecture
8	Media Vault & Storage Strategy
9	Scheduler, Queue and Workers
10	Platform Connector Layer
11	Authentication & Security Architecture
12	Database Model - Phase 1
13	Multilingual Architecture
14	Visual Design System
15	Web vs Mobile Capability Policy
16	Infrastructure for Demo and Production
17	Backup, Redundancy and Failover
18	Analytics & AI Roadmap
19	Commercial Model
20	Estimated Development Effort & Costs
21	Current Implementation Status
22	Next Development Steps
23	Repository & Deployment Strategy
24	Risks and Technical Principles

1. Executive Summary

The planned product is a professional SaaS for creators and creator agencies. Its primary purpose is to remove repetitive manual work from content distribution. A creator or agency should upload or reference a media asset once, select the connected platforms, define dates, times and platform-specific options, and allow the system to execute each publication automatically.

The product is not intended to be a public content platform. It is a private management, scheduling, distribution and analytics system. Media is accessible only to authorized users inside a workspace and is distributed to external platforms selected by the user.

POC success criterion

Upload one video once, schedule it for one or more connected platforms, execute the publication automatically, and display a verifiable status such as Scheduled, Publishing, Published or Failed.

2. Product Vision & Core Problem

The initial client is an agency managing multiple creators. The operational pain is not the lack of content; it is the effort required to distribute a large content library across multiple monetization platforms. Teams naturally prioritize the platform producing the most revenue and often neglect secondary platforms because manual uploading, captioning, scheduling and tracking is repetitive.

- Centralize media into one operational library (Vault).
- Distribute the same asset to multiple platforms without repeating the upload workflow manually.
- Schedule different dates/times per platform.
- Track whether each asset has already been used on each platform.
- Allow agencies to manage multiple creators inside one workspace.
- Evolve later into unified analytics, AI recommendations, thumbnail generation, captions, hashtags, team management and CRM.

3. Proof of Concept Scope

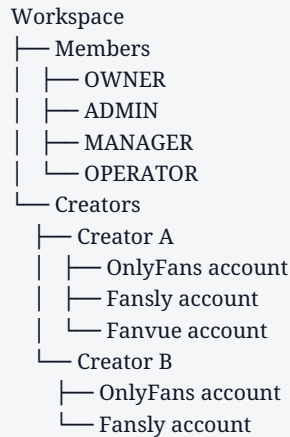
The demo is intentionally narrower than the final SaaS. It must prove the core technical thesis and be credible enough to support an investment decision.

Included in functional demo	Deferred until investment approval
Private login	Full billing/subscription system
One workspace / agency	Advanced pricing tiers
1-2 test creators	Full employee administration
Basic Vault / media selection	Large-scale storage optimization
Scheduler and job queue	Advanced analytics
At least one real platform connector	AI strategy engine
Publication status and errors	Automatic thumbnail generation

Multilingual UI foundation	Full CRM / messaging center
Professional visual foundation	Mobile store production release

4. Target Users and Multi-Tenant Model

The architecture should support both individual creators and agencies without creating two separate products.



A creator plan is simply a workspace with one primary creator. An agency plan is the same architecture with multiple creators, more members and broader permissions.

5. Core User Flows

5.1 Agency flow

1. Owner signs in to the workspace.
2. Owner selects or creates a creator profile.
3. Creator platform accounts are connected and authorized.
4. Media is uploaded, indexed or selected from the Vault.
5. The user selects one or more target platforms.
6. The user defines date, time, caption, price/PPV or audience settings where supported.
7. The scheduler creates one publication job per platform.
8. Workers execute jobs independently.
9. The system records success, failure, retries and platform response identifiers.

5.2 Example

Asset	Platform	Schedule	Status
video_023.mp4	OnlyFans	Mon 6:00 PM	Scheduled
video_023.mp4	Fansly	Mon 8:00 PM	Scheduled

video_023.mp4	Fanvue	Tue 10:00 AM	Scheduled
video_023.mp4	Platform 4	Tue 2:00 PM	Scheduled

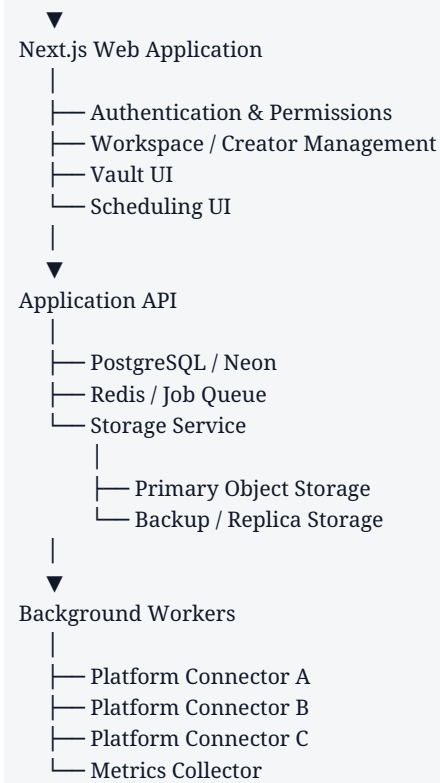
6. Technology Stack

Layer	Recommended technology	Purpose
Frontend / Web App	Next.js 16 + React	Responsive dashboard and web experience
Language	TypeScript	Shared type safety across frontend/backend
Database	PostgreSQL on Neon	Users, workspaces, creators, schedules, metadata, metrics
ORM / Data contract	Prisma 8 contract workflow	Typed database access and schema control
Styling	Tailwind CSS	Responsive design system and UI components
Icons	Lucide React	Consistent minimal iconography
i18n	next-intl	English, Portuguese, Spanish and French
Queue / Cache	Redis / Upstash or managed Redis	Jobs, retry, rate limiting and coordination
Media Storage	Cloudflare R2 / S3-compatible storage	Vault media and thumbnails
Backup Storage	Backblaze B2 or equivalent	Independent copy, versioning, object lock
Workers	Node.js / TypeScript services	Background uploads, publishing and metrics collection
Monitoring	Sentry + structured logs	Errors, tracing and operational visibility
Source Control	Private GitHub repository	Version control, rollback and CI/CD

7. System Architecture

User / Agency

|



Heavy media transfer and background work should not run inside the main web request path. The application API coordinates work; workers perform it.

8. Media Vault & Storage Strategy

The Vault is the central content library. A media item should exist once in the logical library even if it is scheduled to five platforms. Five publication jobs reference the same asset rather than creating five permanent copies.

8.1 Storage modes

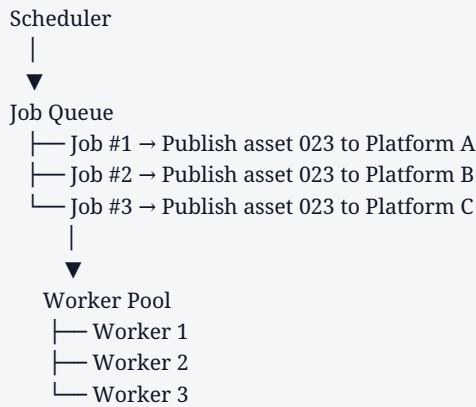
- Cloud Vault: media stored in object storage and available even when the user's computer is offline.
- Local Library (future): optional desktop agent indexes a user-selected local folder and uploads the source file only when required.
- Platform reference (where technically possible): media already stored on a connected platform may be referenced without creating an unnecessary duplicate.

8.2 Commercial storage policy

Storage should be quota-based, not unlimited. An initial commercial concept discussed was approximately 200 GB included in a creator plan, with additional 100 GB / 500 GB / 1 TB add-ons. Agency storage can be pooled at workspace level.

9. Scheduler, Queue and Workers

A worker is a background process that executes a queued job independently from the web application. This is essential for scheduled publishing, retries and large uploads.



- Workers can scale horizontally without scaling the frontend.
- A failed platform does not block other platform jobs.
- Retry strategy can use delays such as 2 minutes, 10 minutes, then a final retry.
- Workers should store platform response IDs and error details for auditability.

10. Platform Connector Layer

Each external platform must be isolated behind a connector interface. This prevents one platform's API or authentication changes from destabilizing the rest of the product.

```

Connector interface
├─ connectAccount()
├─ refreshAuthorization()
├─ uploadMedia()
├─ createPost()
├─ schedulePost()
├─ getPublicationStatus()
└─ fetchMetrics()
  
```

Where an official API/OAuth exists, it should be preferred. Where no official public API exists, integration feasibility, terms, authentication stability, 2FA and anti-automation controls must be evaluated before commercializing that connector.

Critical dependency

The application, queue, storage and scheduling layers are fully controllable. Connector reliability is partly dependent on what each external platform allows and exposes.

11. Authentication & Security Architecture

Security is a first-class requirement because the platform will manage private media, external account authorizations and agency operations.

- Email + password authentication.
- Passwords stored only as strong hashes; Argon2 is installed for the project.

- Two-factor authentication using TOTP.
- Recovery codes stored as hashes.
- Email verification tokens.
- Password reset tokens with expiration and one-time use.
- Session table with token hashes, expiration, client type, IP, user agent and revocation.
- Login attempt recording for abuse detection and rate limiting.
- Audit logs for sensitive actions.
- Account deletion request workflow.
- HttpOnly, Secure and SameSite cookies for browser sessions.
- Secrets and platform tokens stored encrypted and never committed to Git.
- Public signup can be disabled through environment flags during the private demo.

12. Database Model - Phase 1

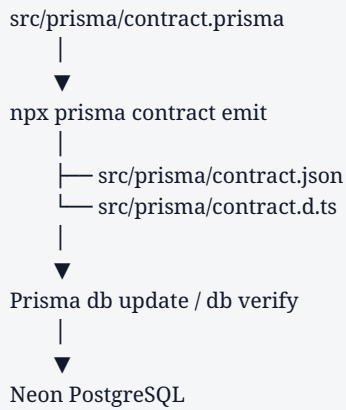
The Phase 1 data model is focused on identity, multi-tenancy, creators and authentication. Media, schedules, publications and platform accounts will be introduced in later phases.

Model	Purpose
User	Human account that can sign in
Workspace	Commercial account / agency / creator workspace
WorkspaceMember	User membership and role inside a workspace
Creator	Creator/model managed by the workspace
Session	Authenticated session and device/client information
TwoFactorMethod	TOTP configuration for one user
RecoveryCode	Hashed one-time recovery codes
EmailVerificationToken	Email verification flow
PasswordResetToken	Secure password recovery
LoginAttempt	Security monitoring and brute-force controls
AuditLog	Sensitive action history
AccountDeletionRequest	Account/data deletion workflow

12.1 Current Prisma / Neon state

The project is using Prisma 8's contract-based workflow. The schema is authored in ``src/prisma/contract.prisma``, emitted to ``contract.json`` and ``contract.d.ts``, and the database is verified against the contract. A new clean Neon PostgreSQL database was created after early schema experiments, preventing legacy objects from contaminating the working database.

Current contract workflow



13. Multilingual Architecture

The project base language is English (US). All code, folders, database identifiers, internal statuses and technical comments are written in English. The interface is multilingual.

Locale	Language
en-US	English (default)
pt-BR	Portuguese
es-ES	Spanish
fr-FR	French

A single page is rendered in different languages through translation keys. Pages are not duplicated. The current routing is locale-based, for example `/en-US/login`` and `/pt-BR/login``.

Implementation rule

No visible UI text should remain hard-coded in English. Every user-facing label, message, button and validation message should come from the translation dictionaries.

14. Visual Design System

The selected direction is clean, premium and futuristic rather than gamer-style or heavily neon.

- Dark graphite background rather than pure black.
- Deep neutral surfaces and thin borders.
- Blue/cyan/violet accent used selectively for attention and calls to action.
- Minimal line icons.
- Rounded controls with modern proportions.
- High contrast where action is required; lower saturation elsewhere.
- Responsive desktop and mobile layouts using shared components.

- Micro-interactions should be subtle and fast.

The demo currently includes a premium dark login screen. Branding remains intentionally neutral until the final product name is approved.

15. Web vs Mobile Capability Policy

The system should support a shared backend while allowing different capabilities for web and mobile clients.

Client	Policy
WEB	Full professional management feature set, subject to product policy
MOBILE	Conservative feature set designed to satisfy the stricter mobile-store policy baseline

The backend should enforce capability differences based on authenticated client type rather than merely hiding buttons in the UI. This allows the mobile apps to remain focused on management, scheduling, analytics and notifications while the web application can support broader private operational workflows where appropriate.

16. Infrastructure for Demo and Production

16.1 Private demo

- Existing domain + private subdomain.
- Private GitHub repository.
- Neon PostgreSQL.
- Minimal managed hosting.
- Small object storage allocation only for test media.
- Public registration disabled; demo users created manually.
- Optional Cloudflare Access / email allow-list in front of the subdomain.

16.2 Professional launch

Component	Suggested service
DNS / WAF / CDN	Cloudflare
Next.js frontend	Vercel or Render
API / backend services	Render or equivalent
Background workers	Render / Railway / container service
PostgreSQL	Neon
Redis / queue	Upstash / managed Redis
Object storage	Cloudflare R2
Independent backup	Backblaze B2

Monitoring	Sentry + centralized logs
------------	---------------------------

A dedicated server is not required at launch. The architecture should scale each service independently instead of concentrating the application, database, media and workers on one large server.

17. Backup, Redundancy and Failover

Backup and high availability are separate concerns.

- Database: managed backups, point-in-time recovery when available, periodic external export.
- Media: primary object storage plus independent backup provider.
- Backup provider should support object versioning and, where practical, Object Lock/immutability.
- Production failover may use a storage abstraction that can switch reads/writes to a secondary provider if the primary is unavailable.
- Failover health checks should validate real storage operations, not just ICMP/ping.
- Failback requires reconciliation of objects written during the outage.

Important

Provider redundancy is not the same as backup. If the application accidentally deletes an object, a redundant primary storage can replicate the deletion. Versioned or immutable backup is what enables recovery.

18. Analytics & AI Roadmap

Analytics and AI are intentionally deferred until the distribution engine is reliable, but the data model should be designed to capture metrics from the beginning.

- Revenue by platform.
- Revenue by creator.
- Revenue by media asset.
- Views, likes, purchases, conversion and subscriber impact where available.
- Best-performing platform for each content type.
- Best-performing publishing time.
- Top videos and underperforming content.
- AI-generated captions, hashtags and thumbnail concepts.
- AI recommendations for which content to publish where and when.
- Future assisted scheduling: AI proposes a weekly plan, human approves it.

19. Commercial Model

Licensing should be workspace-based, with creator and agency plans using the same technical platform.

Concept	Illustrative structure
Creator	1 creator, limited connected platforms, base storage quota
Creator Pro	1 creator, higher limits and advanced features
Agency Starter	Several creators, shared workspace storage, multiple staff

	members
Agency Pro	Higher creator count, more team seats and analytics
Agency Business	Custom creator count, storage and support
Storage add-on	Additional 100 GB / 500 GB / 1 TB
AI / analytics add-on	Advanced intelligence as a premium feature

An early pricing idea discussed was approximately US\$39/month for an individual plan with about 200 GB included, with additional storage sold separately. Final pricing should be based on actual storage, connector and support costs after the POC.

20. Estimated Development Effort & Costs

20.1 Development effort

Milestone	Estimated effort
POC - core proof with one real connector	Approximately 25-40 focused hours
Stronger MVP with 2-3 connectors	Approximately 70-100 hours total
Additional platform connector	Roughly 8-20+ hours depending on API/authentication complexity

20.2 Initial infrastructure

For the private demo, infrastructure can remain near zero to a few tens of dollars per month. A professional initial production environment was estimated broadly around US\$100-250/month before large-scale media storage or expensive third-party connector services.

Storage is the main variable cost. Commercial plans should therefore include explicit quotas and paid add-ons rather than unlimited storage.

21. Current Implementation Status

Area	Status
Local project	Created in C:\Demo\creator-platform
Git	Initialized and pushed to private GitHub repository
Main branch	main tracking origin/main
Next.js	Running locally
i18n	Working for en-US, pt-BR, es-ES, fr-FR

Login UI	Premium dark visual created
Database	Clean Neon PostgreSQL database connected
Prisma	Prisma 8 contract workflow validated
Core data model	User, Workspace, WorkspaceMember, Creator, Session
Security data model	2FA, recovery codes, email verification, password reset, login attempts, audit/deletion structure planned/being finalized
Real authentication	Not yet connected to login form
Platform connector	Not yet implemented
Vault	Not yet implemented
Scheduler / worker	Not yet implemented

22. Next Development Steps

- Finalize Phase 1 contract with all authentication/security models and verify against Neon.
- Create a controlled demo user seed/creation script.
- Implement password hashing with Argon2.
- Implement login API/action and session creation.
- Connect the existing login UI to real authentication.
- Implement TOTP 2FA enrollment and verification.
- Create the first protected internal dashboard.
- Add Workspace and Creator management UI.
- Define Platform and PlatformAccount models.
- Implement the first real connector.
- Add media Vault, scheduler, queue and worker execution.
- Prepare the private subdomain deployment for investor demonstration.

23. Repository & Deployment Strategy

The demo and production product should remain the same codebase. Domain changes must be configuration changes, not code rewrites.

- Use environment variable `APP\_URL` rather than hard-coded domains.
- Use `.env` for local secrets; never commit credentials.
- Keep `.env.example` with safe variable names only.
- Use private GitHub repository and meaningful commits.
- The demo can run on a temporary subdomain; after approval, point the final product domain to the same application.
- OAuth callbacks, cookie domains, CORS and webhooks must reference environment-based URLs.

24. Risks and Technical Principles

Principle / Risk	Decision
External platform changes	Isolate each platform behind a connector layer
Huge video libraries	Object storage, quotas, local-first option later
Simultaneous publishing	Queue + horizontally scalable workers
Single point of failure	Separate frontend, DB, storage, queue and workers
Credential exposure	OAuth/tokens preferred; encrypt secrets; no plaintext passwords
Mobile policy differences	Backend-enforced client capability policy
Internationalization debt	All UI text through translation keys from day one
Premature infrastructure cost	POC on minimal services; scale after investment
Premature feature expansion	Prove distribution engine first
Schema churn	Contract emit + dry-run + verify before destructive DB operations

Appendix A - Current Project File Structure (Simplified)

```
creator-platform/
├─ app/
│  └─ [locale]/
│     └─ page.tsx
│     └─ login/
│        └─ page.tsx
├─ i18n/
│  └─ request.ts
│  └─ messages/
│     └─ en-US.json
│     └─ pt-BR.json
│     └─ es-ES.json
│     └─ fr-FR.json
├─ src/
│  └─ prisma/
│     └─ contract.prisma
│     └─ contract.json
│     └─ contract.d.ts
│     └─ db.ts
├─ .env
├─ .env.example
├─ next.config.ts
├─ package.json
└─ prisma.config.ts
```

Appendix B - Project Working Rules

- Base language: English (US) for code, database, files and default UI.
- Supported UI languages from the beginning: English, Portuguese, Spanish and French.
- Full-file replacements preferred during guided implementation; avoid fragment-by-fragment edits.
- Before database changes: emit contract, run dry-run, inspect for destructive operations, apply, then verify.
- Do not use production-scale infrastructure before investor approval.
- The demo should remain private and accessible only to explicitly authorized users.
- Design should remain clean, futuristic, premium and easy on the eyes.
- The POC is successful when the core distribution workflow works end-to-end.

END OF PRE-DOCUMENTATION